PANDAS QUICK REFERENCE

READ AND INSPECT A DATASET

Action	Code
Read CSV	`df = pd.read_csv("file.csv")`
Read Excel	`df = pd.read_excel("file.xlsx")`
First rows	`df.head()`
Last rows	`df.tail()`
Shape	`df.shape`
Column names	`df.columns`
Data types	`df.info()`
Summary stats	`df.describe()`
Index	`df.index`

LOCATE AND SELECT

Action	Code
Single column	`df['col']`
Multiple columns	`df[['col1','col2']]`
Row by label	`df.loc[5]`
Row by position	`df.iloc[5]`
Cell	`df.loc[2,'col']`
Filter	`df[df['col'] > 10]`
Multiple conditions	`df[(df['A']>5) & (df['B']<3)]`

DATA CLEANING

Action	Code
Check missing	`df.isnull().sum()` / `df.isna().sum()`
Drop NA	`df.dropna()`
Fill NA	`df.fillna(0)`
Fill with mean/median	`df['Age'].fillna(df['Age'].median())`
Rename	`df.rename(columns={'old':'new'})`
Drop columns	`df.drop(['c1','c2'], axis=1)`
Apply function	`df['new'] = df['col'].apply(lambda x: x**2)`
Elementwise	`df.applymap(str.upper)`
Convert type	`df['col'] = df['col'].astype(float)`
Check dtypes	`df.dtypes`

USEFUL OPERARTIONS

Action	Code
Sort	`df.sort_values('col')`
Multi sort	`df.sort_values(['A','B'], ascending=[True,False])`
Unique values	`df['col'].unique()`
Count unique	`df['col'].nunique()`
Duplicated check	`df.duplicated()`
Drop duplicates	`df.drop_duplicates()`
To datetime	`df['date'] = pd.to_datetime(df['date'])`
Extract year	`df['date'].dt.year`
Extract month	`df['date'].dt.month`

GROUPING AND AGGREGATION

Action	Code
Group mean	`df.groupby('cat')['val'].mean()`
Group sum	`df.groupby('cat')['val'].sum()`
Group count	`df.groupby('cat')['val'].count()`
Group min/max	`df.groupby('cat')['val'].min()` / `.max()`
Multi agg	`df.groupby(['A','B']).agg({'C':'sum','D':'mean'})`
Pivot table	`pd.pivot_table(df, values='val', index='A', columns='B', aggfunc='mean')`
Value counts	`df['col'].value_counts()`

COMBINING DATA

Action	Code
Merge	`pd.merge(df1, df2, on='key')`
Concat rows	`pd.concat([df1, df2], axis=0)`
Concat cols	`pd.concat([df1, df2], axis=1)`
Melt	`df.melt(id_vars=['id'], value_vars=['A','B'])`
Pivot	`df.pivot(index='id', columns='var', values='val')`

HANDY ONE-LINERS

Task	Code	Brief Note
Normalize column	`df['col'] = (df['col'] - df['col'].mean()) / df['col'].std()`	Standardize to mean=0, std=1

Min-Max scale	`(df['col'] - df['col'].min()) / (df['col'].max() - df['col'].min())`	Scale to [0,1]
Bin ages	`pd.cut(df['age'], bins=[0,18,35,60,100], labels=['child','young','adult','senior'])`	Categorize continuous values
Dummy variables	`pd.get_dummies(df['category'], prefix='cat')`	One-hot encode categorical
Replace values	`df['col'].replace({'old':'new'})`	Map values quickly
String ops	`df['col'].str.lower()`	Vectorized string methods
Crosstab	`pd.crosstab(df['A'], df['B'])`	Frequency table of two vars
Correlation matrix	`df.corr()`	Numeric feature relationships
Top N values	`df.nlargest(5, 'col')`	Largest values
Bottom N values	`df.nsmallest(5, 'col')`	Smallest values
Frequency normalize	`df['col'].value_counts(normalize=True)`	Relative frequencies
Conditional replace	`df.loc[df['col']>100, 'col'] = 100`	Cap values
Combine columns	`df['full'] = df['first'] + " " + df['last']`	String concatenation
Map categories	`df['grade'] = df['score'].map({90:'A',80:'B'})`	Map values to labels